# Tarefa 19: Equação do Calor 2D em GPU com OpenMP e Perfilamento com nsys

Israel Soares de Castro Filho
Matrícula: 20250070780

## 1 Introdução

O programa resolve a equação do calor em 2D por diferenças finitas explícitas (estêncil de 5 pontos) em uma grade $n \times n$ (exemplo `heat.c` do tutorial de programação de GPUs com OpenMP). O objetivo da tarefa é executar o cálculo na GPU com diretivas OpenMP, observar a movimentação de dados entre o *host* (CPU) e o dispositivo (GPU) e perfilar o programa com o `nsys` para diferentes tamanhos de problema.

## 2 Metodologia

- **Ambiente:** computador pessoal com GPU NVIDIA RTX 4070 Super, compilador `gcc` 15 com suporte a *offload* para NVIDIA (OpenMP) e `nsys` para o perfilamento. Os testes não foram feitos no NPAD porque a fila estava cheia.
- **Paralelização:** o laço do estêncil usa `target teams loop` e `collapse(2)` para distribuir as iterações entre os times de threads da GPU.
- **Movimentação de dados:** a cláusula `map(tofrom: u, u_tmp)` fica dentro de `solve()`, então os dois vetores vão para a GPU e voltam a cada passo de tempo.
- **Parâmetros:** $n \in \{500, 1000, 2000, 4000, 8000\}$, 100 passos de tempo, uma execução por tamanho.
- **Medições:** o tempo de *solve* vem do próprio programa. O tempo do kernel e o tempo e o volume das cópias vêm do `nsys` (relatórios `cuda_gpu_kern_sum`, `cuda_gpu_mem_time_sum` e `cuda_gpu_mem_size_sum`).

Compilação:

```
gcc -O3 -fopenmp -foffload=nvptx-none -fcf-protection=none -fno-stack-protector -o
    tarefa19 tarefa19.c -lm
```

Perfilamento (para cada $n$):

```
nsys profile --trace=cuda,osrt --sample=none -o perfis/n$N ./tarefa19 $N 100
```

# 3 Resultados

A Tabela 1 mostra os tempos medidos. O tempo de *solve* inclui o *overhead* do profiler e a inicialização da GPU, pois cada tamanho foi uma execução única.

Tabela 1: Tempos por tamanho de grade (100 passos).

| $n$ | Solve (s) | Kernel (s) | Cópia HtoD (s) | Cópia DtoH (s) | Dados (GB) |
|---|---|---|---|---|---|
| 500 | 0,233 | 0,041 | 0,022 | 0,024 | 0,8 |
| 1000 | 0,419 | 0,082 | 0,093 | 0,100 | 3,2 |
| 2000 | 1,294 | 0,281 | 0,376 | 0,407 | 12,8 |
| 4000 | 4,938 | 1,300 | 1,602 | 1,589 | 51,2 |
| 8000 | 19,349 | 5,331 | 7,255 | 6,313 | 204,8 |

A Tabela 2 compara, por passo de tempo, o custo do kernel com o das cópias.

Tabela 2: Custo por passo de tempo: kernel contra cópias.

| $n$ | Kernel (ms) | Cópias (ms) | Cópias / kernel | Cópias no tempo de GPU |
|---|---|---|---|---|
| 500 | 0,41 | 0,46 | 1,1 | 53 % |
| 1000 | 0,82 | 1,94 | 2,4 | 70 % |
| 2000 | 2,81 | 7,83 | 2,8 | 74 % |
| 4000 | 13,00 | 31,91 | 2,5 | 71 % |
| 8000 | 53,31 | 135,68 | 2,5 | 72 % |

# 4 Análise

- **As cópias dominam o tempo.** Para $n \geq 1000$, cerca de 70 % do tempo de GPU ocupada é transferência de dados, e cada passo gasta cerca de 2,5 vezes mais tempo copiando do que calculando. Isso acontece porque o `map(tofrom)` em `solve()` copia os dois vetores nos dois sentidos a cada passo, embora `u` só seja lido e `u_tmp` só seja escrito.

- **O volume de dados confere com o esperado.** São $4 \cdot n^2 \cdot 8 \cdot$ passos bytes, por exemplo 204,8 GB para $n = 8000$. A taxa de cópia ficou perto de 16 GB/s, limitada pelo barramento entre CPU e GPU.

- **O custo cresce com $n^2$.** Ao dobrar $n$, os tempos de kernel e de cópia crescem cerca de 4 vezes para grades grandes (de 4000 para 8000, o *solve* passou de 4,9 s para 19,3 s).

- **Grades pequenas têm custo fixo alto.** Para $n = 500$, o kernel e as cópias somam só 87 ms dos 233 ms de *solve*. O restante é inicialização da GPU (criação do contexto, cerca de 88 ms) e alocações.

# 5 Conclusão

O programa executa corretamente na GPU e o erro L2 diminui com o refinamento da grade. O `nsys` mostrou que, na versão testada, a movimentação de dados é o principal gargalo: as cópias a cada passo ocupam cerca de 70 % do tempo de GPU e confirmam o volume previsto pelo código. A escolha da diretiva de paralelização também teve grande impacto no desempenho. Manter os dados na GPU durante o laço de tempo é o próximo passo para reduzir esse custo.

# A Código-Fonte

```c
#include <stdlib.h>
#include <stdio.h>
#include <math.h>

#include <omp.h>

#define PI acos(-1.0)
#define LINE "--------------------\n"

/**
 * PARAMETROS DA COLETA DE DADOS (substituem o antigo tarefa19.sh)
 *
 * ./tarefa19          -> varre os tamanhos abaixo, REPS repeticoes cada,
 *                        e grava os tempos em ARQUIVO_CSV
 * ./tarefa19 <n> <nsteps> -> uma unica execucao (como no codigo original),
 *                        util para perfilar com nsys
 */
#ifndef REPS
#define REPS 3                    // repeticoes por tamanho
#endif
#ifndef NSTEPS
#define NSTEPS 100                // passos de tempo na varredura
#endif
#define ARQUIVO_CSV "resultados_tarefa19.csv"

static const int TAMANHOS[] = {500, 1000, 2000, 4000, 8000}; // grade n x n
#define N_TAMANHOS ((int)(sizeof(TAMANHOS) / sizeof(TAMANHOS[0])))

/**
 * DEFINICOES DE FUNCAO
 */
void executa(int n, int nsteps, double *solve_time, double *total_time, double *
    norm_out);
void initial_value(const int n, const double dx, const double length, double *
    restrict u);
void zero(const int n, double * restrict u);
void solve(const int n, const double alpha, const double dx, const double dt, const
    double * restrict u, double * restrict u_tmp);
double solution(const double t, const double x, const double y, const double alpha,
    const double length);
double l2norm(const int n, const double * restrict u, const int nsteps, const double
    dt, const double alpha, const double dx, const double length);

/**
 * MAIN
 */
int main(int argc, char *argv[]) {

  printf("Dispositivos OpenMP: %d\n", omp_get_num_devices());

  double solve_time, total_time, norm;

  // Execucao unica, como no codigo original
  if (argc == 3) {

    // Problem size, forms an nxn grid
```

```
52      int n = 1000;
53
54      // Number of timesteps
55      int nsteps = 10;
56
57      // Set problem size from first argument
58      n = atoi(argv[1]);
59      if (n < 0) {
60        fprintf(stderr, "Error: n must be positive\n");
61        exit(EXIT_FAILURE);
62      }
63
64      // Set number of timesteps from second argument
65      nsteps = atoi(argv[2]);
66      if (nsteps < 0) {
67        fprintf(stderr, "Error: nsteps must be positive\n");
68        exit(EXIT_FAILURE);
69      }
70
71      executa(n, nsteps, &solve_time, &total_time, &norm);
72      return 0;
73    }
74
75    // Varredura de tamanhos e repeticoes
76    FILE *csv = fopen(ARQUIVO_CSV, "w");
77    if (!csv) {
78      perror("Erro ao abrir " ARQUIVO_CSV);
79      return EXIT_FAILURE;
80    }
81    fprintf(csv, "n,nsteps,rep,tempo_solve_s,tempo_total_s,erro_l2\n");
82
83    // Inicializa o contexto da GPU antes das medicoes
84    int dummy = 0;
85    #pragma omp target map(tofrom: dummy)
86    { dummy += 1; }
87
88    for (int k = 0; k < N_TAMANHOS; ++k) {
89      int n = TAMANHOS[k];
90
91      printf("=== n=%d | aquecimento (descartado) ===\n", n);
92      executa(n, NSTEPS, &solve_time, &total_time, &norm);
93
94      for (int rep = 1; rep <= REPS; ++rep) {
95        printf("=== n=%d | rep=%d ===\n", n, rep);
96        executa(n, NSTEPS, &solve_time, &total_time, &norm);
97        fprintf(csv, "%d,%d,%d,%lf,%lf,%E\n", n, NSTEPS, rep, solve_time, total_time,
              norm);
98        fflush(csv);
99      }
100   }
101
102   fclose(csv);
103   printf("Concluido. Resultados em %s\n", ARQUIVO_CSV);
104   return 0;
105 }
106
107
108 // Corpo do main original: uma execucao completa para um n e nsteps
```

```
109 void executa(int n, int nsteps, double *solve_time, double *total_time, double *
        norm_out) {
110 
111   // Start the total program runtime timer
112   double start = omp_get_wtime();
113 
114 
115   //
116   // Set problem definition
117   //
118   double alpha = 0.1;        // heat equation coefficient
119   double length = 1000.0;    // physical size of domain: length x length square
120   double dx = length / (n+1); // physical size of each cell (+1 as don't simulate
        boundaries as they are given)
121   double dt = 0.5 / nsteps; // time interval (total time of 0.5s)
122 
123 
124   // Stability requires that dt/(dx^2) <= 0.5,
125   double r = alpha * dt / (dx * dx);
126 
127   // Print message detailing runtime configuration
128   printf("\n");
129   printf(" MMS heat equation\n\n");
130   printf(LINE);
131   printf("Problem input\n\n");
132   printf(" Grid size: %d x %d\n", n, n);
133   printf(" Cell width: %E\n", dx);
134   printf(" Grid length: %lf x %lf\n", length, length);
135   printf("\n");
136   printf(" Alpha: %E\n", alpha);
137   printf("\n");
138   printf(" Steps: %d\n", nsteps);
139   printf(" Total time: %E\n", dt*(double)nsteps);
140   printf(" Time step: %E\n", dt);
141   printf(LINE);
142 
143   // Stability check
144   printf("Stability\n\n");
145   printf(" r value: %lf\n", r);
146   if (r > 0.5)
147     printf(" Warning: unstable\n");
148   printf(LINE);
149 
150 
151   // Allocate two nxn grids
152   double *u    = malloc(sizeof(double)*n*n);
153   double *u_tmp = malloc(sizeof(double)*n*n);
154   double *tmp;
155 
156   // Set the initial value of the grid under the MMS scheme
157   initial_value(n, dx, length, u);
158   zero(n, u_tmp);
159 
160   //
161   // Run through timesteps under the explicit scheme
162   //
163 
164   // Start the solve timer
```

```
  double tic = omp_get_wtime();
  for (int t = 0; t < nsteps; ++t) {

    // Call the solve kernel
    // Computes u_tmp at the next timestep
    // given the value of u at the current timestep
    solve(n, alpha, dx, dt, u, u_tmp);

    // Pointer swap
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
  }
  // Stop solve timer
  double toc = omp_get_wtime();

  //
  // Check the L2-norm of the computed solution
  // against the *known* solution from the MMS scheme
  //
  double norm = l2norm(n, u, nsteps, dt, alpha, dx, length);

  // Stop total timer
  double stop = omp_get_wtime();

  // Print results
  printf("Results\n\n");
  printf("Error (L2norm): %E\n", norm);
  printf("Solve time (s): %lf\n", toc-tic);
  printf("Total time (s): %lf\n", stop-start);
  printf(LINE);

  // Devolve os valores para a coleta de dados
  *solve_time = toc - tic;
  *total_time = stop - start;
  *norm_out  = norm;

  // Free the memory
  free(u);
  free(u_tmp);

}


// Sets the mesh to an initial value, determined by the MMS scheme
void initial_value(const int n, const double dx, const double length, double *
    restrict u) {

  double y = dx;
  for (int j = 0; j < n; ++j) {
    double x = dx; // Physical x position
    for (int i = 0; i < n; ++i) {
      u[i+j*n] = sin(PI * x / length) * sin(PI * y / length);
      x += dx;
    }
    y += dx; // Physical y position
  }

```

```
}


// Zero the array u
void zero(const int n, double * restrict u) {

  for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
      u[i+j*n] = 0.0;
    }
  }

}


// Compute the next timestep, given the current timestep
void solve(const int n, const double alpha, const double dx, const double dt, const
    double * restrict u, double * restrict u_tmp) {

  // Finite difference constant multiplier
  const double r = alpha * dt / (dx * dx);
  const double r2 = 1.0 - 4.0*r;

  // Loop over the nxn grid
  #pragma omp target teams loop collapse(2) map(tofrom: u[0:n*n], u_tmp[0:n*n])
  for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {

      // Update the 5-point stencil, using boundary conditions on the edges of the
          domain.
      // Boundaries are zero because the MMS solution is zero there.
      u_tmp[i+j*n] = r2 * u[i+j*n] +
      r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
      r * ((i > 0)  ? u[i-1+j*n] : 0.0) +
      r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
      r * ((j > 0)  ? u[i+(j-1)*n] : 0.0);
    }
  }
}


// True answer given by the manufactured solution
double solution(const double t, const double x, const double y, const double alpha,
    const double length) {

  return exp(-2.0*alpha*PI*PI*t/(length*length)) * sin(PI*x/length) * sin(PI*y/length)
      ;

}


// Computes the L2-norm of the computed grid and the MMS known solution
// The known solution is the same as the boundary function.
double l2norm(const int n, const double * restrict u, const int nsteps, const double
    dt, const double alpha, const double dx, const double length) {

  // Final (real) time simulated
  double time = dt * (double)nsteps;
```

```
275
276   // L2-norm error
277   double l2norm = 0.0;
278
279   // Loop over the grid and compute difference of computed and known solutions as an
          L2-norm
280   double y = dx;
281   for (int j = 0; j < n; ++j) {
282     double x = dx;
283     for (int i = 0; i < n; ++i) {
284       double answer = solution(time, x, y, alpha, length);
285       l2norm += (u[i+j*n] - answer) * (u[i+j*n] - answer);
286
287       x += dx;
288     }
289     y += dx;
290   }
291
292   return sqrt(l2norm);
293
294 }
```

Código 1: Código-fonte da Tarefa 19 (`tarefa19.c`)